

Seminar 2

Object-Oriented Design, IV1350

Ellen Grönholm
ellengro@kth.se

VT26 v14

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

Usage of AI

The usage of AI assistance in the report is limited to help of environmental issues with the IDE and explaining observed errors when coding. As AI assistance ChatGPT-5.5 (OpenAI) and Gemini 3 Flash (Google) have been used.

Introduction

This report examines how an Object-Oriented Design could look for a scenario of a bike-workshop. For this a domain model (DM), a system sequence diagram (SSD), a design class diagram, and an interaction diagram is made and presented. Lastly these models will be analyzed and criticized.

Requirements Specification for Repair Electric Bike

The following specifications were taken from the "Seminar 1, Analysis : Object-Oriented Design, IV1350" instructions (Leif Lindbäck, 2026).

Requirements Specification for Repair Electric Bike

The scenarios below describe basic flow and alternative flows for servicing bikes, at an electric bike repair workshop.

Basic flow

1. Customer arrives at workshop with bike to repair.
2. Receptionist greets customer and asks for customer's phone number.
3. Customer tells phone number.
4. Receptionist enters customer's phone number.
5. System searches customer registry for customer details (name and email address), and for details about the customer's bike (brand, model and serial number).
6. System presents customer details and details of the customer's bike. This data is known since the customer has consulted the workshop previously.
7. Receptionist asks customer if details of customer and bike are correct.
8. Customer confirms that details are correct.
9. Receptionist asks customer for a description of the problem with the bike.
10. Receptionist enters customer's description.
11. System creates a repair order containing customer details, bike details, problem description and date.
12. Customer waits while a technician inspects the bike.
13. Technician asks system for repair order.
14. System presents repair order.
15. Technician performs diagnostic.
16. Technician enters diagnostic report and proposed repair tasks.
17. System updates repair order, by adding diagnostic report and proposed repair tasks.
18. Receptionist informs customer about diagnostic report, proposed repair tasks, cost for each proposed repair task, and total cost.

19. Customer accepts proposed repair tasks and cost.
20. Receptionist registers that customer accepted repair order.
21. System prints repair order. The printout contains all repair order data, including estimation of when reparation will be completed.
22. Receptionist gives the printed repair order to customer.
23. Customer leaves.

Alternative Flows

- 5a. Unknown phone number
 1. System fails to find customer details.
 2. System tells receptionist that phone number doesn't exist in customer registry.
- 19a. Repair order not accepted
 1. Customer doesn't accept repair order, and doesn't want to consult the bike repair workshop.
 2. Receptionist specifies that repair order is rejected.
 3. System registers that repair order is canceled, but doesn't delete it from the repair order registry.

1 Method

The following workflows were executed to transition from analysis to design:

- **Analysis Review:** The Domain Model and System Sequence Diagram (SSD) were analyzed for consistency.
- **System Startup Design:** A design for the initialization phase (`main` method) was established to handle object creation and layering dependencies.
- **Dynamic Behavior Modeling:** Interaction diagrams (Sequence Diagrams) were drafted to map system operations to specific software objects, tracing both the basic flow and alternative flows of the "Repair Electric Bike" scenario.
- **Static Structure Modeling:** A comprehensive Class Diagram was derived directly from the message pathways established in the interaction diagrams.

The workflow used when creating the model was to first create a DM by finding all objects and connecting them, the SSD is made using the different flows and the DM. As the SSD is created, the DM is updated to include necessary methods and attributes, and sometimes the associations had to be redrawn.

When this was completed, the instructions, DM and SSD were used to create a design class diagram. This expands on the DM by putting all classes into layers. The layers include at least *Model*, *View* and *Controller* as specified by the *MVC* model. The *Layer* patterns introduce more layers and thus requires one to reflect over what other layers are needed for the project. In this report it was decided to introduce the layers *startup*, *dbhandler*, and *printSystem* as the *model* layer should only include business logic.

- The *startup* layer starts and creates all databases and initiates the view and controller.
- The *dbhandler* takes care of all repositories that work with data handling. In a complete model there would also be a *database* layer which stores the data, however this is disregarded for the model, only leaving the knowledge that the data-layer should only be connected to the *dbhandler* layer.
- the *printSystem* is needed for the printer to be connected to the whole system as it is not part of neither the data-base or the business logic. This layer connects both creates the receipt and sends it to the printer.

For the interaction diagram all previous diagrams and the instructions are used to connect all relevant classes through methods so the code can support specific operations and follow the workflow needed. The SSD specifies what the system is supposed to do based on outside interactions with the system. The interaction diagrams display how the system responds to each interaction. For each interaction diagram it is specified which interaction it represents. In total the following interactions are created for this report:

- Start Up of System
- Find Customer
- Add Problem List
- Add Diagnostic Report
- Add Proposed Repairs
- Set Order Status (responsible for printing)

All these diagrams are based on a view and a controller method that are not defined other than having specific methods needed to call the methods of the system.

2 Result

2.1 Architectural Overview

The system architecture strictly implements the *Model-View-Controller (MVC)* and *Layered* patterns to enforce separation of concerns.

- **View Layer:** Consists of a simple **View** class responsible for displaying output to the technician/receptionist and capturing user inputs. It contains zero business logic.
- **Controller Layer:** Consists of a single **Controller** class that acts as a coordinator. It receives requests from the View and delegates them to the Model layer, holding no domain state or business rules itself.
- **Model Layer (Domain):** Contains the core business rules and entities of the workshop (e.g., **RepairOrder**, **Bike**, **Technician**). It operates independently of the View layer.
- **Start-Up Layer:** Consists of a simple **Main** class responsible for starting up the program. This should only happen once, as it initiates everything.
- **Data Base Handler Layer:** This layer can also be called integration, but is decided to be kept as **dbhandler** as it exclusively works with the data base. It contains two data base registries, and classes to produce their respective DTO.
- **Print System Layer:** Contains the class **printToReceipt** which will convert the Repair Order to a string and then send it to the printer to be printed.

2.2 Class Diagram

The static class diagram illustrates the structural layout of the software system. Every class, interface, method, and association depicted here is derived directly from the message transfers modeled in the interaction diagrams.

the *dbhandler*. In the data package the objects 'customer' and 'bike' would exist, where the bike is part of the customer. This allows easy possibility to change properties of this data, for example is a customer can have more than one bike. If that was the case, another registry might be added, but this is not of importance in this report.

2.3 Interaction Diagrams

The interaction diagrams model the dynamic behavior of the entire system execution. This includes the initialization process starting from the **main** method, the primary "Repair Electric Bike" flow, and all alternative flows (e.g., handling missing customer records or invalid part IDs).

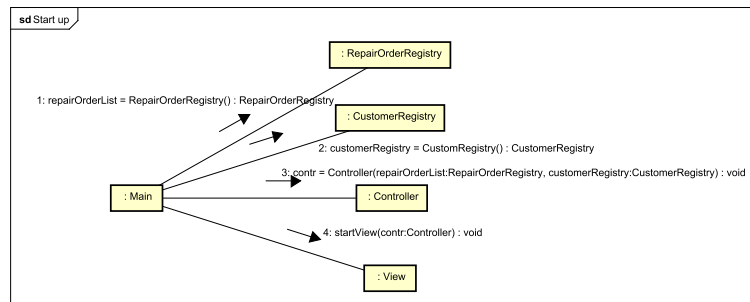


Figure 2: An Interaction Diagram showing how the system is started.

The start up interaction diagram (Figure 2) presents how the system would be started for the first time. the main method would create new registries and then give their locations to the control layer so that it can find them when necessary. The same is done regarding the controller and the view, where the created controller is sent to the view so that the view knows how to interact with the system.

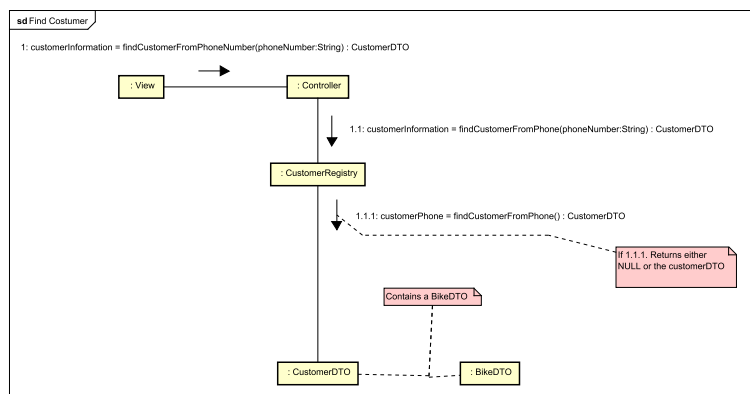


Figure 3: An Interaction Diagram showing the finding of a customer.

Since no data should be accessed directly by the controller, the controller needs to go through the customer registry to find the data necessary to fulfill the request from the view to gather data about the customer and their bike (Figure 3). As this data should not be changed in this scenario, the code actually displays a customer DTO instead of bringing back the original customer data location. If no customer is found using the phone number, the program will return a NULL value.

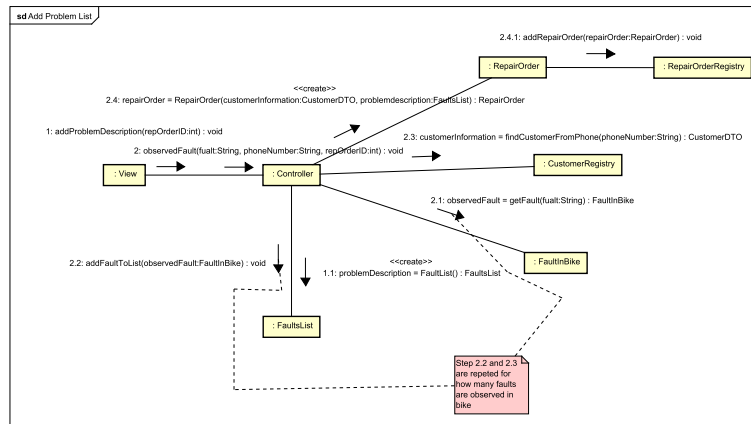


Figure 4: An Interaction Diagram showing how the the customers observed problems are registered.

For the customer to add a list of problems with the bike, the customer will be able to pick an amount of faults that will be added in the faults list. Then a new instance of 'RepairOrder' should be created to which both the customer and the list of problems mentioned by the customer should be attached to. This is thus done in the same workflow. The whole workflow is displayed in Figure 4.

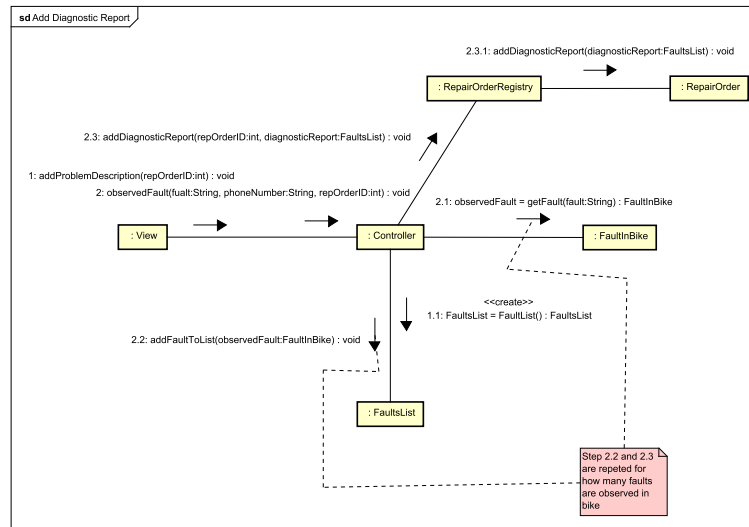


Figure 5: An Interaction Diagram showing how the observed faults by the technician are registered.

Figure 5 is similar to figure 4, but has the difference of not needing to create a new Repair Order. Instead it works with the repair order created in figure 4.

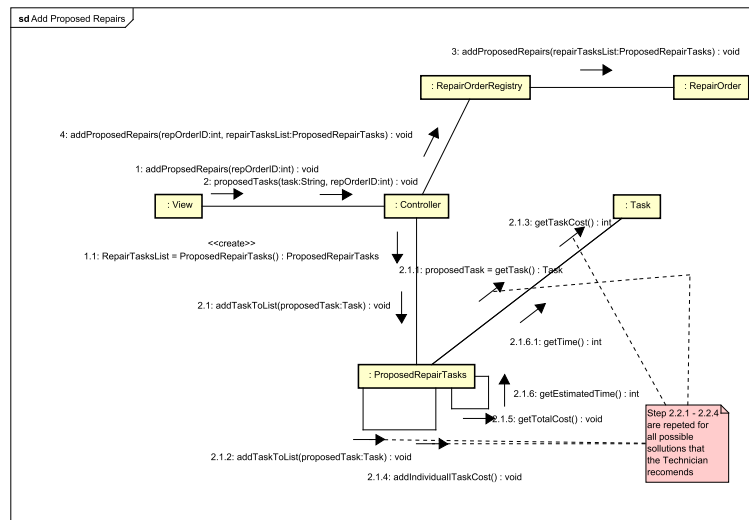


Figure 6: An Interaction Diagram showing how the proposed repairs are registered.

While Figure 6 has the same purpose as Figure 4 and 5, the 'ProposedRepairTasks' contains more than the list of proposed repair tasks. It thus

has direct access to Task as it needs more information. The class 'ProposedRepairTasks' also will call on itself to create all necessary attributes.

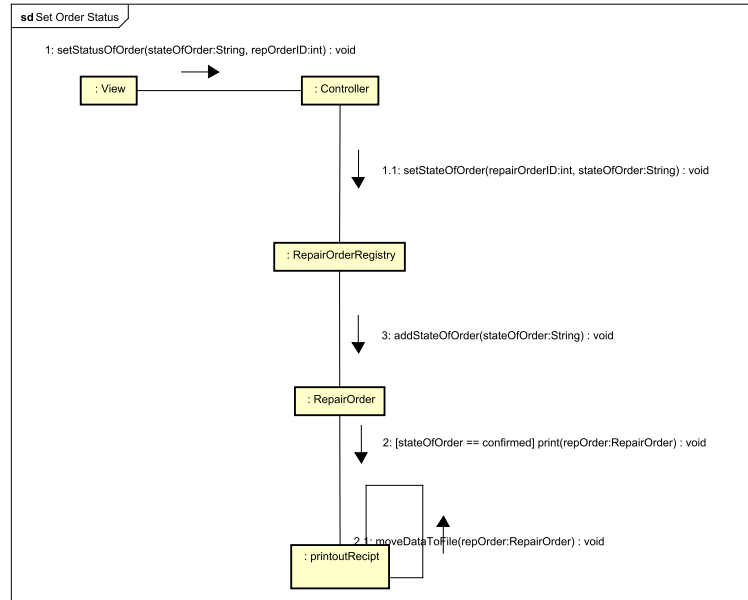


Figure 7: An Interaction Diagram showing how the status of the repair report is set.

It is not specified how the receptionist and customer should get to know that the technician has done the check-up of the bike (No matter how, it would be decided by adding that flow to Figure 6) but when it is done, the receptionist will show the technicians additions to the repair order to the customer and get an cancellation or acceptance as answer (Figure 7). The receptionist updates the system with this information. If the customer accepts the task report, the system will print an receipt that the customer can take with them.

3 Discussion

3.1 Evaluation of Architectural Patterns

An evaluation of the architecture confirms strict boundary compliance:

- **MVC Integrity:** The **Controller** functions purely as a pass-through layer, delegating immediately to domain models. The Model layer possesses no references to, or awareness of, the View layer, facilitating complete user interface pluggability.

- **Layer Cleanliness:** Communication flows strictly downward. Higher layers call lower layers, but lower layers never call upward, avoiding circular dependencies.

3.2 Object-Oriented Design Principles

Achieving a grade of Distinction relies on the following structural qualities:

3.2.1 Cohesion

High cohesion is ensured by having a single, well-defined purpose for each class and layer. Different types of logic are separated into distinct layers; `startUp` for initialization, `dbhandler` for data management, `printSystem` for external hardware interaction, and the `model` for business logic. Furthermore, no class contains functions or attributes unrelated to its core purpose. For example, the model layer is strictly reserved for business logic, such as the `RepairOrder` and lists of tasks/faults, rather than UI or data storage.

One thing that was debated during the implementation was where to contain the `tasks` information about price and mostly where the price total would be. This could either be kept in the `RepairOrder`, or in the `TaskList`. It was decided to keep the price within the `TaskList` as it is closer to the tasks, and the repair order already contains enough information. In general, simple data structures are kept as attributes within relevant classes rather than being scattered across the system.

3.2.2 Coupling

Low coupling is achieved by limiting the dependencies between classes, ensuring that a change in one part of the system has minimal impact on others. To achieve this, the `controller` does not access data directly; it must go through a registry (`CustomerRegistry` and `RepairOrderRegistry`) to retrieve information. And then, instead of passing around original data locations or complex objects, the design uses Data Transfer Objects (DTOs). This ensures that layers like the `view` only receive the data they need without gaining "direct possibility to get to everything" or to edit anything. By following the Layer pattern, the code ensures that the model has no knowledge of the view, and the controller acts only as a limited intermediary and handles no business logic itself.

3.2.3 Encapsulation

Encapsulation is maintained by creating barriers between informational data and methods controlled by the public interface. To do this, methods and attributes are kept private unless they are explicitly required by other classes. For example have getters and setters been implemented instead of creating

global/public variables. This controls and ensures that data is read without being modified, unless okay to do. Returning DTOs instead of original object references is another way that the integrity of the data is kept. By using DTO's the core objects can't be changed and thus prevent external classes from accidentally changing the program's state. The design ensures that even if a technician's view or a database implementation changes, the rest of the system remains unaffected because the internal implementation details are hidden behind the registry and controller interfaces.

3.3 Technical and State Integrity Details

3.3.1 Avoidance of Primitives (Primitive Obsession)

Addressing the seminar's assessment criteria which encourage using objects over primitive data as much as possible, the faults and tasks will be mentioned. The problems seen in the bike are not simply stored as raw text; they are encapsulated within a `FaultInBike` class, which allows the system to manage these observations as distinct domain entities within a structured `FaultsList`. For this the `Faults` need to be established. While it is not yet implemented, it has been thought about to create a new enumerate object of the faults, instead of having them stored as strings in a list that can be chosen.

A similar thing has been done for the repair tasks in the `Task` and `ProposedRepairTasks` classes. While the individual components of a task, such as a cost or an ID, might eventually be represented by numerical types, they are wrapped within the `Task` object to ensure they always carry the necessary context of a specific repair operation. The `ProposedRepairTasks` object then manages these entities, handling the internal logic of the repair proposal rather than forcing the controller or view to manually process raw lists of primitive values.

While it was established to be enough for this assignment, possible expansions would be to create enumerate classes of the states for example. This report also left the price as an int, but this could be changed to a `Object 'Amount'` which could incorporate more than only the price; if expansion to other counties happen, the currency might be necessary information as well.

The design further avoids primitive obsession by utilizing `CustomerDTO` to group customer information. Instead of passing individual strings for a customer's name, phone number, and email address across layers, the code uses an object to maintain the integrity of the customer's identity while providing a read-only snapshot for the view.

3.3.2 Code Cleanliness and Java Conventions

The design avoids the use of unnecessary static attributes and methods, ensuring that system state is properly managed through object instances rather than global variables. All class, method, and variable names strictly follow standard Java naming conventions as specified in the course guidelines.

4 References

- Lindbäck, L. (March 23 2026) *A First Course in Object-Oriented Development: A Hands-On Approach*.

5 Re-submission

The UML has not been updated that much:

- a CustomerDTO is returned instead of string when finding the customer (in figure 3).
- the Controller was calling the View, as I did not understand how the view works, and I was imagining that was needed to change the users view. After talking to class mates and understanding how a view can be implemented, these arrows have just been removed as they were unnecessary. Instead DTO's are returned now.

Other than that, parts of the method have been rewritten and the discussion has been restructured. It has also been expressed how the print layer works (in both the method and results as feedback was given that this was unclear).